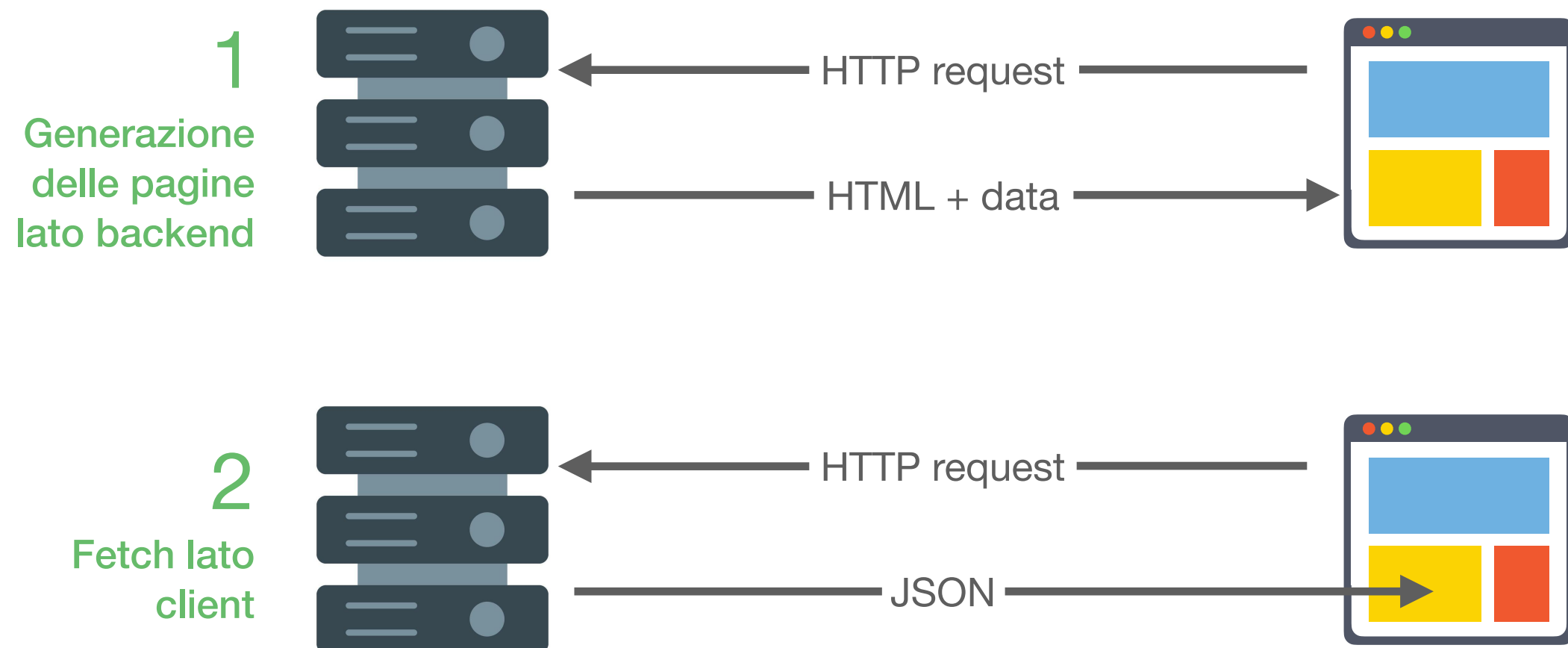


Programmazione Web e Mobile con Laboratorio

Lezione 14 - AJAX
(Chiamate asincrone e fetch)

Pagine dinamiche

- Due approcci per generare contenuti dinamicamente
- Differenza: dove e quando vengono generate le pagine e i dati



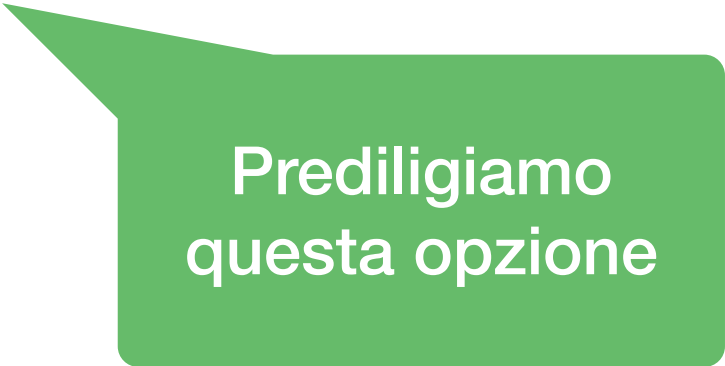
Generazione lato backend

- Il server crea l'intera pagina HTML con i dati già inclusi
- Il client riceve la pagina già pronta e la visualizza
- Esempio: PHP, EJS, Pug
- Vantaggi: meno richieste al server, funziona anche senza JavaScript nel browser
- Svantaggi: ogni modifica richiede un refresh della pagina

Fetch lato client

- Il server invia solo i dati
- Il client costruisce la pagina dinamicamente
- Esempio: JavaScript + API REST
- Vantaggi: il client non ricarica tutta la pagina, ma aggiorna solo le parti necessarie
- Svantaggi: dipende da JavaScript, gestione più complessa (separata per frontend e backend)

Fetch lato client



Prediligiamo
questa opzione

- Il server invia solo i dati
- Il client costruisce la pagina dinamicamente
- Esempio: JavaScript + API REST
- Vantaggi: il client non ricarica tutta la pagina, ma aggiorna solo le parti necessarie
- Svantaggi: dipende da JavaScript, gestione più complessa (separata per frontend e backend)

AJAX

- Sta per **A**synchronous **J**avaScript **A**nd **X**ML
- Oggi al posto di XML si usa quasi sempre JSON
- È una **tecnica di sviluppo web** che consente di inviare e ricevere dati da un server in modo asincrono
- Permette di creare pagine dinamiche aggiornando solo specifiche porzioni del contenuto (come elenchi o tabelle) senza dover ricaricare l'intera pagina

Flusso di una richiesta AJAX

- L'utente compie un'azione (ad esempio clicca un bottone) che fa scattare una richiesta AJAX
- La richiesta viene inviata da JavaScript al server in modo asincrono, cioè senza bloccare l'esecuzione del resto della pagina
- Il server riceve la richiesta e la elabora, magari interrogando un database
- Quando il server ha completato l'elaborazione, invia una risposta al client in formato JSON
- JavaScript riceve la risposta e la elabora, aggiornando il DOM senza fare un refresh completo

Esempio con fetch

- Node.js

```
const express = require("express");

const app = express();
const port = 3000;

app.use(express.static("public"));

app.get("/api/message", (req, res) => {
  res.json({ message: "Questo messaggio è stato inviato dal server!" });
});

app.listen(port, () => {
  console.log("Server attivo su http://localhost:" + port);
});
```


Esempio con fetch

- HTML

```
<!DOCTYPE html>
<html lang="it">

<head>
  <meta charset="UTF-8" />
  <title>AJAX</title>
  <script src="script/script.js"></script>
</head>

<body>
  <h1>Richiesta AJAX</h1>
  <button id="sendBtn">Invia richiesta</button>
  <p id="response"></p>
</body>

</html>
```

Esempio con fetch

- JavaScript

```
window.onload = function () {
  const btn = document.getElementById("sendBtn");
  const responseP = document.getElementById("response");

  btn.addEventListener("click", async () => {
    try {
      const res = await fetch("/api/message");
      if (!res.ok) {
        throw new Error("Errore nella risposta");
      }
      const data = await res.json();
      responseP.textContent = data.message;
    }
    catch (err) {
      console.error("Errore:", err);
      responseP.textContent = "Errore nella richiesta";
    }
  });
};
```

Async e await

- **async**: indica che una funzione restituisce una **promise**
 - Una **promise** è oggetto che rappresenta il risultato futuro di un'operazione asincrona (valori: pending, fulfilled, rejected)
- **await**: sospende l'esecuzione fino a quando la promise è risolta
 - Può essere usato solo dentro una funzione async
 - Non blocca tutto il programma, ma solo la funzione corrente
 - *await* è zucchero sintattico per *.then()*

Async e await

- Nell'esempio...

```
window.onload = function () {  
  const btn = document.getElementById("btn");  
  const responseP = document.getElementById("response");  
  
  btn.addEventListener("click", async () => {  
    try {  
      const res = await fetch("/api/message");  
      if (!res.ok) {  
        throw new Error("Errore nella risposta");  
      }  
      const data = await res.json();  
      responseP.textContent = data.message;  
    }  
    catch (err) {  
      console.error("Errore:", err);  
      responseP.textContent = "Errore";  
    }  
  });  
};
```

Aspetta che la richiesta HTTP
sia completata e arrivi la
risposta dal server

Aspetta che venga letto e
decodificato il corpo della
risposta

express.json()

- Quando un client invia dati al server, ad esempio tramite una form, questi dati sono inseriti nel corpo della richiesta
- Express.js non può accedere direttamente a req.body se non è presente un middleware per decodificare quel contenuto
- `express.json()` è un middleware di Express.js che consente di leggere e interpretare il body delle richieste HTTP
- Alternativa: `express.urlencoded()` per `application/x-www-form-urlencoded` (il formato predefinito per le form)

Esempio con `express.json()`

- Node.js

```
const express = require("express");
const app = express();
const port = 3000;

// Middleware per interpretare JSON
app.use(express.json());

app.use(express.static("public"));

app.post("/api/submit", (req, res) => {
  const { nome, email, eta, messaggio, citta } = req.body;
  const response = {
    messaggio: "Grazie per la tua iscrizione, " + nome + "!<br>" + "Email: " +
email + "<br>" + "Età: " + eta + "<br>" + "Città: " + citta + "<br>" + "Messaggio: "
+ messaggio
  };
  res.json(response);
});

app.listen(port, () => {
  console.log("Server attivo su http://localhost:" + port);
});
```

Esempio con express.json()

- HTML

```
...  
<form id="form" action="/api/submit" method="POST">  
  
  <label for="nome">Nome:</label><br>  
  <input type="text" id="nome" name="nome" required />  
  
  <label for="email">Email:</label><br>  
  <input type="email" id="email" name="email" required />  
  
  <label for="eta">Età:</label><br>  
  <input type="number" id="eta" name="eta" required />  
  
  <label for="messaggio">Messaggio:</label><br>  
  <textarea id="messaggio" name="messaggio" required></textarea>  
  
  <label for="citta">Città:</label><br>  
  <input type="text" id="citta" name="citta" required />  
  
  <button type="submit">Invia</button>  
  
</form>  
...
```


Esempio con express.json()

- JavaScript

```
...
const data = {
  nome: document.getElementById("nome").value,
  email: document.getElementById("email").value,
  eta: document.getElementById("eta").value,
  citta: document.getElementById("citta").value,
  messaggio: document.getElementById("messaggio").value
};
try {
  const res = await fetch("/api/submit", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data)
  });
  const result = await res.json();
  risposta.textContent = result.messaggio;
}
catch (error) {
  console.error("Errore:", error);
  risposta.textContent = "C'è stato un errore durante l'invio";
}
...
```


Parametri della fetch

- `method: "POST"`
specifica il metodo utilizzato nella richiesta HTTP
- `headers: { "Content-Type": "application/json" }`
imposta l'intestazione per indicare che i dati sono in formato JSON
- `body: JSON.stringify(data)`
converte i dati in una stringa JSON per inviarli nel corpo della richiesta

E i framework?

- I framework più popolari come React e Vue utilizzano il **Virtual DOM**, una rappresentazione basata su JavaScript dell'interfaccia utente reale che aiuta a determinare quali parti devono essere aggiornate quando i dati cambiano
- In molti casi, l'overhead introdotto da questi strumenti supera i benefici
- Domanda: quale ha un tempo di First Meaningful Paint migliore tra:
 1. un file HTML grezzo da 8,5 MB con il testo completo di ~30.000 tweet
 2. una pagina in React renderizzata lato client con 1 tweet

E i framework?

- I framework più popolari come React e Vue utilizzano il **Virtual DOM**, una rappresentazione basata su JavaScript dell'interfaccia utente reale che aiuta a determinare quali parti devono essere aggiornate quando i dati cambiano
- In molti casi, l'overhead introdotto da questi strumenti supera i benefici
- Domanda: quale ha un tempo di First Meaningful Paint migliore tra:
 1. un file HTML grezzo da 8,5 MB con il testo completo di ~30.000 tweet
 2. una pagina in React renderizzata lato client con 1 tweet
- Risposta: secondo Lighthouse vince il primo

Casi reali

- **GitHub** ha rimosso jQuery dall'applicazione web, preferendo metodi nativi del browser e componenti personalizzati
- **Netflix** ha eliminato React dal caricamento frontend predefinito, utilizzando JS vanilla per il codice lato client e riducendo i tempi di caricamento del 50%
- **Airbnb** ha ridotto il peso delle pagine e introdotto l'uso efficiente di risorse e il lazy loading delle immagini

Alcune buone pratiche



- Prediligere siti statici o architetture serverless quando possibile
- Usare tecnologie semplici (JS, HTML e CSS vanilla) quando possibile, senza framework pesanti
- Non includere librerie solo per funzionalità minime
- Ridurre peso e numero di risorse e caricare solo ciò che serve
- Evitare tracciamenti e analytics superflui

Esercizio

- Funzionalità 1: invio colore
 - L'utente specifica un colore in una form
 - Il colore viene inviato al server tramite POST
 - Il server appende il colore a un file di testo
- Funzionalità 2: recupero ultimo colore
 - L'utente clicca un bottone che invia una richiesta GET al server
 - Il server restituisce l'ultimo colore salvato nel file
 - Il client usa il colore ricevuto per ricolorare lo sfondo della pagina